Occ Bilayer + SAM Repository: Deep Architecture, Hyperparameter, and Reproduction Guide

This document is the implementation companion to the Bilayer/SAM theory review. It explains repository inheritance, exact source paths, model/config resolution, tensor flow, all released model and training hyperparameters, annotation handling, weight conversion, paper–code discrepancies, commands, and acceptance tests. The reviewed Occ commit is `7d8c5fe`; RSPrompter release is `7c676fe`.

1. Repository Identity and Full Lineage

The official Occ repository is a modified framework, not a small standalone module. Its README credits RSPrompter and MMDetection. RSPrompter supplies the Mask R-CNN + SAM learned-prompt application architecture. Occ adds the occlusion-specific dataset path, second mask decoder, auxiliary loss, residual guidance, and task configs.

Table 1: Which repository supplies each layer.

Source	Version / reviewed branch	Implementation responsibility
Occ	Official commit `7d8c5fe`	Occlusion data/transforms, bilayer RoI/mask head, dual decoder, auxiliary loss, converters, configs and checkpoints.
RSPrompter	Release commit `7c676fe`	Mask R-CNN + SAM automatic prompting, model wrappers, scale splitter, prompt MLP and runner integration.
MMDetection	Vendored 3.2.0	RPN, assignment/sampling, box head, datasets, transforms, evaluator and engine.
MMPretrain	Vendored 1.1.1	ViT-SAM backbone and registry integration.
Transformers / SAM	4.38.1 / `facebook/sam-vit-large`	Pretrained ViT-L, positional embedding and mask-decoder tensors.
PEFT	0.8.2	LoRA injection and base-weight freezing.
MMCV	2.1.0	Low-level OpenMMLab operations and runtime support.

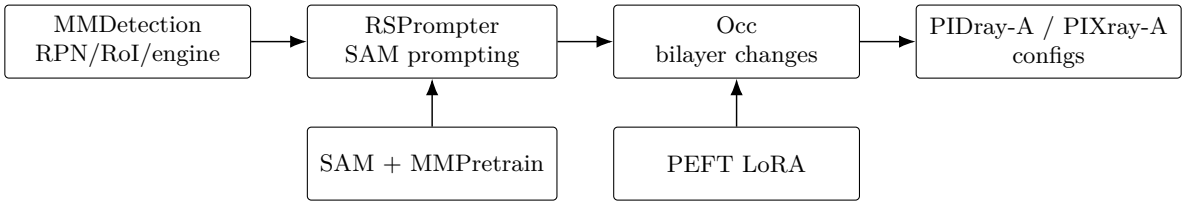


Figure 1: Repository ancestry and configuration endpoint.

2. Configuration Inheritance

`configs/occlusion/pid.py` inherits `base.py`. The base file declares `RSPrompterAnchor`, Mask R-CNN/RPN/RoI defaults, SAM prompt/mask modules, data/evaluator defaults and a long training schedule originally suited to RSPrompter. The PIDray file overrides the backbone, feature aggregator, custom Occ RoI/mask heads, data paths/classes/transforms, schedule, optimizer wrapper and checkpoint rule.

Table 2: Base-to-PIDray override map.

Area	Base RSPrompter value	PIDray/Occ override
Backbone	Hugging Face SAM wrapper, inherited multi-hidden-state path	MMPretrain SAM ViT-L with LoRA; single feature.
Feature aggregation	Selects/aggregates hidden layers	`OcFeatureAggregator`: identity for one feature.
RoI/mask head	RSPrompter prompt head and one SAM decoder	Occ prompt head and two decoders with residual guidance.
Mask target	Full-image target mask	Full-image target plus full-image occluder mask.
Dataset	Remote-sensing example dataset	`OcCocoDataset`; PIDray-A 12 classes.
Transforms	Standard mask transforms	Occ variants transform target and occluder masks together.
Schedule	300 epochs in base	50 PIDray epochs; 60 PIXray epochs.
Checkpoint	Best box and mask metrics in base	PIDray override selects best box mAP only and keeps one.

3. Recommended Source Reading Order

Table 3: Files required to understand the entire released experiment.

Order	Path / source	What to extract
1	`readme.md`	Pinned environment, SAM conversion, data/checkpoints and commands.
2	`configs/occlusion/base.py`	Inherited RSPrompter detector, RPN/RoI and runtime.
3	`configs/occlusion/pid.py`	ViT-L/LoRA, Occ heads, classes, transforms, schedule and paths.
4	`mmdet/occlu/models.py`	RSPrompterAnchor, SAM wrappers, simple FPN and prompt MLP.
5	`mmdet/occlu/occlu.py`	Occ dataset/transforms, `bo_masks`, bilayer head, dual decoder and losses.
6	`convert_model_weights.py`	Duplication of original decoder weights into occluder decoder.
7	`convert_occlusion_annotation.py`	Heuristic custom-data occlusion annotation.
8	RSPrompter release	Which model behavior predates Occ.
9	Vendored `mmdet/`, `mmpretrain/`, `tools/`	Framework versions, runner and evaluator entry points.

4. Complete Released Tensor Path

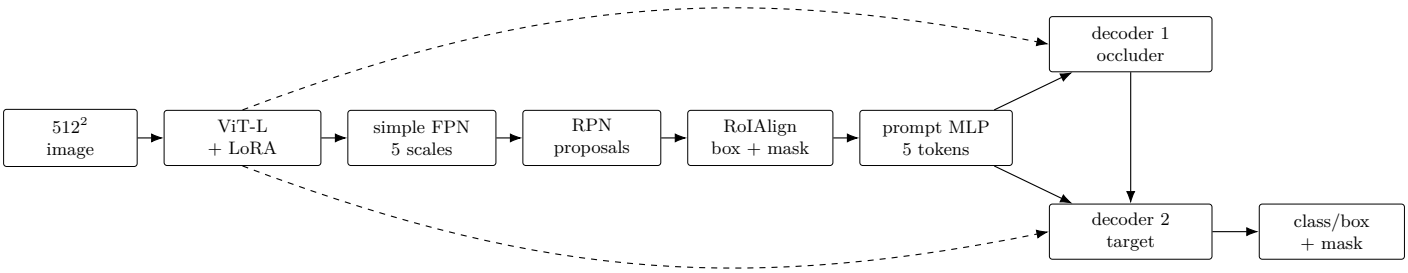


Figure 2: Code-level data flow from image to automatic full-image instance mask.

5. SAM ViT-L and LoRA Hyperparameters

Table 4: Exact released backbone/adaptation values.

Property	Value	Implementation consequence
Input / patch	512 / 16	$32 \times 32 = 1,024$ visual tokens.
Embedding / depth	1,024 / 24 blocks	SAM ViT-L.
Heads / MLP	16 / 4,096	Large transformer capacity.
Window/global	window 14; global indices 5/11/17/23	Four global-attention blocks.
Position	absolute + relative	Fixed-grid spatial reasoning.
Output neck	256 channels	One $256 \times 32 \times 32$ image embedding.
LoRA target	every `qkv`	Attention projections adapted.
LoRA values	rank 16; alpha 32; dropout 0.05; no bias	Approx. 1,572,864 code-derived LoRA parameters.
Trainability	base ViT frozen; LoRA trainable	PEFT enforces parameter-efficient adaptation.
Decoder flag	`decoder_freeze=False`	Prompt-side modules and both decoders train.

6. Feature Pyramid, RPN, and Box Hyperparameters

`OcFeatureAggregator` asserts a single feature and returns it unchanged. `RSSimpleFPN` creates four scales from that feature and max-pools a fifth. This is a scale splitter, not a top-down multi-backbone FPN.

Table 5: Scale splitter for a 512-pixel input.

Level	Grid	Stride	Construction	Channels
P_2	128^2	4	Two transposed convs, $256 \to 128 \to 64$	256 after lateral/output convs
P_3	64^2	8	One transposed conv, $256 \to 128$	256
P_4	32^2	16	Identity	256
P_5	16^2	32	2×2 max-pool	256
P_6	8^2	64	Stride-2 max-pool of P_5	256

Table 6: RPN and RoI settings inherited/resolved by the PIDray config.

Component	Values	Notes
RPN anchors	scales 4, 8; ratios 0.5/1/2; strides 4/8/16/32/64	Six anchors/location; ratio-1 sizes 16/32 through 256/512.
RPN assignment	pos 0.7; neg 0.3; min pos 0.3; low-quality match	Sample 256; positive fraction 0.5.
Train proposals	pre-NMS 2,000; max 1,000; NMS 0.7	No minimum box size.
Test proposals	pre-NMS 1,000; max 1,000; NMS 0.7	Same NMS threshold.
Extra FPN position	normalized sine encoding	Added to each pyramid feature before RoI processing.
Box RoI	RoIAlign 7; strides 4/8/16/32	Two FC layers, width 1,024.
Box prediction	12 classes; class-specific regression	Coder stds 0.1/0.1/0.2/0.2.
RoI assignment	pos/neg IoU 0.5; sample 256; positive fraction 0.25	Ground truth added as proposals.
Test output	score 0.05; NMS 0.5; max 100	Mask threshold 0.5.

7. Prompt and Dual-Decoder Implementation

Positive mask RoIs have shape $N_+ \times 256 \times 14 \times 14$. The prompt MLP is: stride-2 3×3 conv + BN + ReLU, flatten, linear to 256, ReLU, linear 256, ReLU, and a final linear to $256 \times 2 \times 5$. Sine/pair collapse produces five 256-D sparse tokens. A learned no-mask embedding becomes the 32×32 dense prompt.

Table 7: Expected tensor checkpoints.

Checkpoint	Expected shape	Purpose
SAM image embedding	$B \times 256 \times 32 \times 32$	Repeated for each RoI.
Mask RoI feature	$N_+ \times 256 \times 14 \times 14$	Input to prompt MLP.
Sparse prompts	$N_+ \times 1 \times 5 \times 256$	Proposal-specific SAM prompt.
Dense prompt	$N_+ \times 256 \times 32 \times 32$	Expanded no-mask embedding.
Decoder token state	$N_+ \times 1 \times 10 \times 256$	IoU token + mask tokens + prompts.
Low-resolution masks	$N_+ \times 1 \times 128 \times 128$ per decoder	Later resized to loss target size.
Target/occluder masks	$N_+ \times 512 \times 512$	Full-image masks selected by assigned GT index.

`OcPrompterAnchorRoIPromptHead._mask_forward` clones mask features, calls the mask head with `if_bo=True`, receives occluder prediction plus refined image/point states, then calls again with `if_bo=False` and residual states. `OCSamMaskDecoder` performs the residual additions only on the target branch.

$$L_{\text{mask}} = \text{BCE}(\widehat{M}, M), \qquad L_{\text{bo}} = 0.25\ \text{BCE}(\widehat{O}, O).$$

Both predictions are interpolated from 128^2 to 512^2. Unlike standard Mask R-CNN, the target builder selects full-image masks by assigned ground-truth index rather than cropping a proposal-relative 28-pixel target.

8. Decoder Weight Conversion

`convert_model_weights.py` loads the Hugging Face PyTorch SAM state dictionary. Every key beginning `mask_decoder.` is deep-copied to a parallel `mask_decoder_bo.` key. Consequently, both decoders begin from identical pretrained tensors and then optimize independently.

Before training, assert:

- both decoder namespaces have identical initial tensor counts and shapes;
- no unexplained missing/unexpected keys remain;
- both decoders have trainable parameters and nonzero gradient norms;
- the source weight is a PyTorch binary, because the converter expects `torch.load`.

9. Occlusion Data Implementation

`OcCocoDataset` expects `bg_object_segmentation` whenever a target segmentation exists. `OcLoadAnnotations` creates parallel bitmap masks. Custom flip, resize, random crop, batch pad and packing classes keep both mask types aligned.

Table 8: Released PIDray-A data pipeline.

Step	Transform	Values/behavior
1	Load image	Float32.
2	`OcLoadAnnotations`	Boxes, labels, target masks, occluder masks.
3	`OcRandomFlip`	Probability 0.5; both mask types.
4	Random resize	Base 512^2; ratio range 0.1–2.0; keep ratio.
5	`OcRandomCrop`	Absolute 512^2; recompute boxes; allow negative crop.
6	Filter annotations	Minimum box width/height 10^{-5}; filter by mask.
7	`OcPackDetInputs`	Maps `gt_bo_masks` to `bo_masks`.
8	Batch pad	Fixed 512^2; pad target and occluder masks with 0.

Fallback hazard. If occluder-mask length does not match valid indices, the packer substitutes target masks. This avoids a crash but corrupts the auxiliary task. Replace it with a hard assertion for research experiments.

Manual release versus converter. The paper's PIDray-A/PIXray-A are manually annotated. `convert_occlusion_annotation.py` is an automatic approximation for custom COCO datasets: it tests pairs with box IoU above 0.05, clips the other object's true mask inside the target box, unions regions, keeps more than 20 pixels, and simplifies long contours. Give converter output a different dataset name and hash.

10. Classes and Dataset Paths

PIDray class order is: Baton, Pliers, Hammer, Powerbank, Scissors, Wrench, Gun, Bullet, Sprayer, HandCuffs, Knife, Lighter. The config expects `annotations/train_occ.json`, `annotations/test_occ.json`, `train/`, and `test/`.

PIXray uses 15 classes: Bat, Battery, Dart, Fireworks, Gun, Hammer, Knife, Lighter, Pliers, Pressure vessel, Razor blade, Saw blade, Scissors, Screwdriver, Wrench. Its config changes class count/order, data paths and maximum epochs to 60.

11. Optimizer, Schedule, Runtime, and Paper Discrepancy

Table 9: Complete released PIDray training values.

Item	Released value
Batch/loading	2 images/GPU; 8 workers/GPU; persistent workers; no declared accumulation.
Optimizer	AdamW, LR 10^{-4}, weight decay 0.05.
Precision	AMP wrapper, float16.
Warm-up	Linear, 50 iterations, start factor 0.001 (effective 10^{-7}).
Scheduler	Cosine annealing from epoch 1 to 50; minimum 10^{-7}.
Training	50 epochs PIDray-A; validation every 3 epochs.
Checkpoint	Interval 1; keep 1; save last; best by `coco/bbox_mAP`.
Distributed	NCCL; `find_unused_parameters=True`.
Test metrics	COCO `bbox` and `segm`.

The paper reports batch 64 on one A800 80 GB. The released config is batch 2/GPU and has no visible accumulation. README says the code was tested on one A800 and eight RTX 4090s. Record the actual effective batch and do not call a config run exact paper optimization unless batch 64 is recovered.

12. Environment and Commands

Table 10: README-tested environment.

Layer	Version
System	Linux; Python 3.10; CUDA 11.8.
PyTorch stack	PyTorch 2.1.2; torchvision 0.16.2; torchaudio 2.1.2.
OpenMMLab	MMCV 2.1.0; vendored MMDetection 3.2.0; MMPretrain 1.1.1.
SAM/adaptation	Transformers 4.38.1; PEFT 0.8.2; Einops.
Data/evaluation	NumPy 1.24; pycocotools; Shapely; SciPy; terminaltables.

Command forms:

```
bash tools/dist_train.sh configs/occlusion/pid.py N
bash tools/dist_test.sh configs/occlusion/pid.py CHECKPOINT N
```

Visible GPU IDs, commit/config hashes, data/SAM/checkpoint hashes, seed, batch, scheduler and hardware must be recorded outside these templates.

13. Exact Reproduction Procedure

1. Pin Occ/RSPrompter commits and build the documented environment using vendored frameworks.
2. Download `facebook/sam-vit-large` with `use_safetensors=False`; save a PyTorch binary.
3. Run decoder-weight conversion and audit both namespaces.
4. Manifest PIDray-A/PIXray-A: image/annotation counts, class order, mask validity and target/occluder overlays.
5. Copy the official config and change only paths.
6. Evaluate the released checkpoint before training; retain raw predictions and subset metrics.
7. Run one forward/backward batch. Audit tensor shapes, trainable parameters, all losses and gradient norms.
8. Smoke train a representative subset containing normal, multi-item and occlusion images.
9. Resolve effective batch; then perform the full run and select on train-derived validation with a segmentation-aware rule.
10. Evaluate overall, Easy/Hard/Hidden and an exact occlusion-subset manifest.

14. High-Risk Checks

Table 11: Common failures and required controls.

Risk	Control
Wrong framework imported	Print module paths and custom registry entries.
Wrong SAM binary	Verify conversion input and all expected keys.
Occluder masks misaligned	Render before/after flip, resize, crop and pad.
Packer target-mask fallback	Replace with equal-count assertion.
Fully frozen or fully trainable SAM	List base frozen, LoRA trainable, both decoders trainable.
Manual/generated data conflated	Use separate dataset identities and hashes.
Paper/config batch mismatch	Report actual batch, GPUs, AMP and optimizer steps.
Hidden performance overclaimed	Always report Hidden AP.
Box-only checkpoint selection	Predeclare mask-aware validation selection.
RSPrompter contribution misattributed	Separate inherited prompt model from Occ bilayer changes.

15. Verified Links

- Occ official: `github.com/Ryh1218/Occ`.
- RSPrompter release: `github.com/KyanChen/RSPrompter/tree/release`.
- SAM ViT-L: `facebook/sam-vit-large`.
- Upstream: MMDetection 3.2.0; MMPretrain 1.1.1; PEFT.